I worked on two different face swapping techniques: SimSwap and Insightface. I will explain what I did accomplish and learned from each one.

SimSwap:
I trained a 224 input-sized Generator model to run faceswap with: test_one_image.py and test_video_swapsingle.py.
I only got to train for 100,000 total steps, so the results were unrefined. See the Google drive links for face swap results for still image and video results.
Note: For reference, I also show the results of a pretrained 512 input-sized model which shows a more optimal face swapping result on same source face and target face video.

- Source face image to swap in
  - https://drive.google.com/file/d/1mqvynWBaKc4iD7MhWpwLcf7h9XyAF-bH/view?usp=drive_link

- Source target video to swap out face
  - https://drive.google.com/file/d/172Sx7HWPfMIGdjWSg65brOntWMAEA_KO/view?usp=drive_link

- **Results:** Image faceswap results for 100,000 step 224 Generator model
  - https://drive.google.com/file/d/1l5yH_5Vr5tidlRdC2wCPklmBCkqIt3tD/view?usp=drive_link

- **Results:** Video faceswap 100,000 step 224 Generator model
  - https://drive.google.com/file/d/1gjZxyqT6mF6sdw6rdhmnJNIUh3Hs82aS/view?usp=drive_link

- More optimal looking results with pre-trained 512 model
  - https://drive.google.com/file/d/1UI586e-S0xV9cOwHMhMoBNr8uJ4otU1_/view?usp=drive_link

SimSwap code description in github rep:

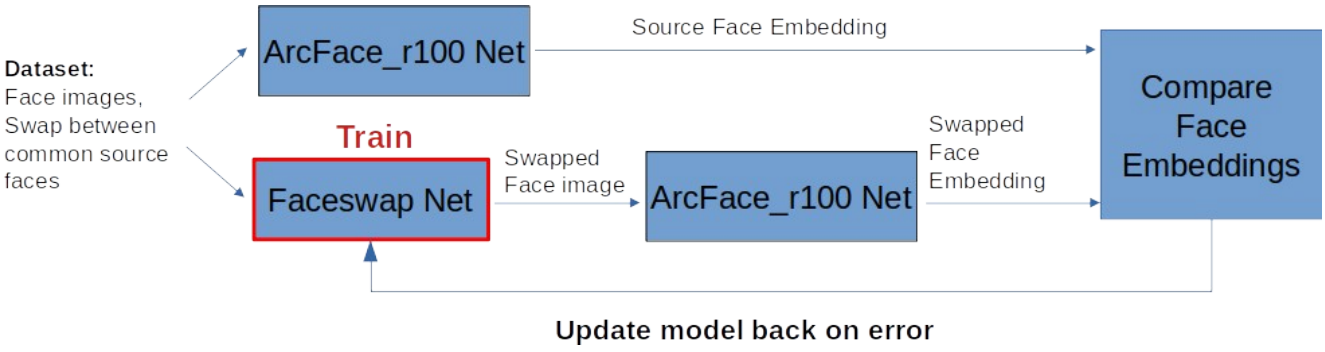
train.ipynb	Jupyter Notebook for training.
SimSwapCustom	
train.py	minor tweaks to get training to execute “resume training of checkpoint.
models/projected_model.py	minor tweaks to get training to execute “resume training of checkpoint.
data/data_loader_Swapping.py	made tweaks to properly read new face aligned data.
insightface_func/ detect_and_align_faces.py	created script to re-align faces in the 250x250 lfw_funnel images into 256x256 images for SimSwap support.
face_detect_crop_single.py	minor tweaks to work with detect_and_align_faces.py
util/reverse2original.py	minor fix to floating point type for running test_video_swapsingle.py

Insightface

I have done some faceswap discovery work for research purposes with Insightface about 3 years ago, so I spent the first 4 days attempting to reactivate that work. The results were underwhelming and the task was a bit too ambitious for a take home test. However I wanted to share for completeness.

I converted a faceswap model into a Pytorch description, with the intention to train the faceswap model solely, as opposed to training a generator and discriminator GAN network. The logic approach is to compare the face embeddings between a pre-trained ArcFace_r100 model with the complementary face embeddings from Faceswap + ArcFace_r100. Based on the loss error, update the weights of the Faceswap model network.

Note: *I was able to train the model with the lfw_funneled dataset but the results were underwhelming. I started to see the face swap crop box go from a "white" box to a "gray" box, which was leaning toward generating facial features, before I decided to pivot to SimSwap.*



Insightface code description in github rep:

insightfaceCustom

train_faceswap.py	A custom training script to train an insightface-based faceswap model directly. Created a custom insightface/recognition/faceswap package based on the other insightface/recognition packages, like arcface_torch. Too big to add here.
custom faceswap dependencies	new faceswap package is similar to arcface_torch: https://github.com/deepinsight/insightface/tree/master/recognition/arcface_torch